FOUNDATION OF JAVA LAB SHEET – 6

Exception Handling — Questions Only

Total Questions: 15 | Easy/Basic: 10 | Real-World: 5

Part A — Easy / Basic Questions

Q1. Handling ArithmeticException

Write a Java program that accepts two integers from the user and divides the first number by the second number. Handle the situation where the user enters 0 as the denominator using try-catch.

Concept: try, catch, ArithmeticException

Q2. Handling ArrayIndexOutOfBoundsException

Create an integer array containing 5 elements. Ask the user to enter an array index and display the corresponding element. Handle the situation where the user enters an invalid index.

Concept: ArrayIndexOutOfBoundsException

Q3. Handling NumberFormatException

Write a Java program that stores a number as a String and converts it into an integer using Integer.parseInt(). Handle the exception if the string does not contain a valid integer.

Concept: NumberFormatException

Q4. Multiple catch Blocks

Write a Java program that:
1. Accepts two numbers from the user.
2. Performs division.
3. Creates an integer array containing five elements.
4. Accepts an index from the user.
5. Displays the corresponding array element.

Handle the following exceptions using separate catch blocks:
• ArithmeticException
• ArrayIndexOutOfBoundsException
• NumberFormatException

Concept: Multiple catch blocks

Q5. Using finally

Write a Java program that accepts two numbers and performs division. Use a finally block to display:

Program execution completed.

The message must be displayed whether an exception occurs or not.

Concept: finally

Q6. Using throw

Write a Java program that accepts a student's marks.

If the marks are:
• Less than 0, or
• Greater than 100

explicitly generate an exception using the throw keyword and display an appropriate error message.

Concept: throw

Q7. Using throws

Create a method named checkAge(int age).

If the age is less than 18, the method should generate an exception. Declare the exception using the throws keyword.

Call the method from main() and handle the exception.

Concept: throws

Q8. Nested try-catch

Write a Java program containing a try-catch block inside another try block.

• The inner try block should perform array access.
• The outer try block should perform division.

Handle both possible exceptions.

Concept: Nested try-catch

Q9. Exception Propagation

Create three methods:

method1()
method2()
method3()

method3() should generate an ArithmeticException.

Allow the exception to propagate from method3() through method2() and method1() and finally handle it in main().

Concept: Exception propagation

Q10. User-Defined Exception

Create a custom exception named InvalidMarksException.

Accept marks from the user.

If the marks are outside the range 0–100, throw the custom exception and display an appropriate error message.

Concept: Custom exception, extends Exception, throw

Part B — Real-World / Application-Based Questions

Q11. Bank Withdrawal System

Create a Java program for a simple bank withdrawal system.

Accept:
• Account balance
• Withdrawal amount

Apply the following rules:
1. Withdrawal amount cannot be negative.
2. Withdrawal amount cannot be greater than the available balance.
3. If the withdrawal amount is greater than the balance, create and throw a custom exception named InsufficientBalanceException.
4. Handle invalid numeric input.
5. Use a finally block to display:

Bank transaction completed.

Concepts: Custom exception, throw, multiple catch, finally, NumberFormatException

Q12. Hospital Patient Age Validation

Create a Java program for hospital patient registration.

Accept:
• Patient name
• Patient age

The valid age range is 0–120.

Create a custom exception named InvalidPatientAgeException.

If the age is outside the valid range, throw the custom exception.

Also handle the situation where the user enters non-numeric input for age.

Concepts: Custom exception, throw, catch, NumberFormatException

Q13. Pharmaceutical Drug Dosage Validation

Create a Java program for validating a patient's drug dosage.

Accept:
• Patient name
• Drug name
• Dosage in mg

The valid dosage range is 1 mg–1000 mg.

Create a custom exception named InvalidDosageException.

If the dosage is less than or equal to 0, or greater than 1000, throw the custom exception.

Also handle invalid numeric input and use a finally block to indicate that dosage validation is completed.

Concepts: Custom exception, throw, try-catch, finally, NumberFormatException

Q14. Online Examination System

Create a Java program for an online examination system.

Accept a student's marks.

Apply the following rules:
0–100 → Valid
< 0 → Invalid
> 100 → Invalid

Create a custom exception named InvalidExamMarksException.

The program should:
1. Accept marks.
2. Validate the marks.
3. Throw the custom exception if marks are invalid.
4. Display PASS if marks are 40 or above.
5. Display FAIL if marks are below 40.
6. Handle invalid numeric input.
7. Use finally to display:

Concepts: Custom exception, throw, catch, finally, NumberFormatException

Q15. Pharmacy Inventory System

Create a Java program for a pharmacy inventory system.

Accept:
• Medicine name
• Available quantity
• Required quantity

Apply the following rules:
1. Quantity cannot be negative.
2. Required quantity cannot be greater than available quantity.
3. If any quantity is negative, throw InvalidQuantityException.
4. If required quantity is greater than available quantity, throw InsufficientMedicineStockException.
5. Handle invalid numeric input.
6. Use multiple catch blocks.
7. Use a finally block to indicate that the inventory transaction is completed.

Concepts: Multiple custom exceptions, throw, multiple catch, finally, NumberFormatException

Concept Coverage

Exception Handling Concept	Questions
try-catch	Q1–Q15
ArithmeticException	Q1, Q4, Q5, Q8, Q9
ArrayIndexOutOfBoundsException	Q2, Q4, Q8
NumberFormatException	Q3, Q4, Q11–Q15
Multiple catch blocks	Q4, Q11, Q15
finally	Q5, Q11–Q15
throw	Q6, Q7, Q10–Q15
throws	Q7
Nested try-catch	Q8
Exception propagation	Q9

Custom/User-defined exceptions	Q10–Q15
Real-world applications	Q11–Q15

Java Programming Lab — Exception Handling